

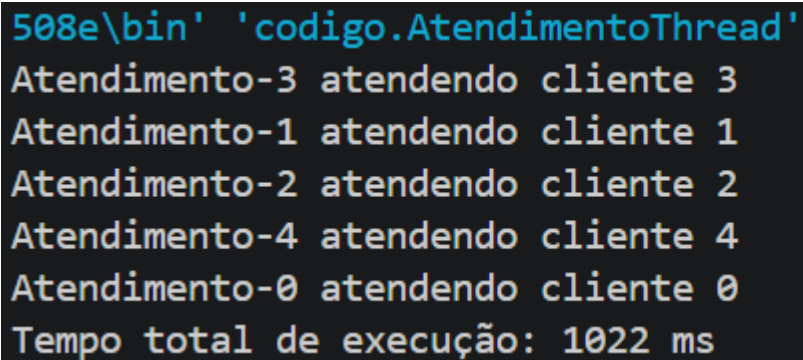
Relatório do Roteiro 1 - Threads em Java

Aluna: Thaís dos Reis Soares

Perguntas de cada parte:

Parte A: O tempo total ficou perto de 1s ou de 5s com 5 atendimentos? Por quê?

O tempo total ficou mais perto de 1 segundo, como ilustra a figura 1. Isso ocorre porque, ao chamar o método `start()` para cada um dos 5 atendimentos, o Sistema Operacional aloca e executa as 5 threads ao mesmo tempo. Como as 5 threads estão rodando ao mesmo tempo, todas elas entram no estado de pausa praticamente no mesmo instante e acordam juntas 1 segundo depois.

A screenshot of a terminal window with a dark background and light-colored text. The text shows the output of a Java program with five threads. Each thread prints a message indicating it is serving a client, with the thread name and client ID. The messages are: 'Atendimento-3 atendendo cliente 3', 'Atendimento-1 atendendo cliente 1', 'Atendimento-2 atendendo cliente 2', 'Atendimento-4 atendendo cliente 4', and 'Atendimento-0 atendendo cliente 0'. The last line shows the total execution time: 'Tempo total de execução: 1022 ms'.

```
508e\bin' 'codigo.AtendimentoThread'  
Atendimento-3 atendendo cliente 3  
Atendimento-1 atendendo cliente 1  
Atendimento-2 atendendo cliente 2  
Atendimento-4 atendendo cliente 4  
Atendimento-0 atendendo cliente 0  
Tempo total de execução: 1022 ms
```

Figura 1

Parte B: Qual das duas classes (Parte A ou B) você poderia fazer herdar de outra classe hoje?

A classe `AtendimentoRunnable` da parte B, porque em Java as classes só podem herdar de uma única classe, e a classe `AtendimentoThread` já herda da classe `Thread`, assim ela não pode herdar de mais nenhuma outra classe. Já a classe `AtendimentoRunnable` implementa a interface `Runnable`, deixando o espaço de herança livre para que ela possa herdar de outra classe.

Parte C: Por que criar uma thread de SO é mais caro do que criar um objeto comum em Java?

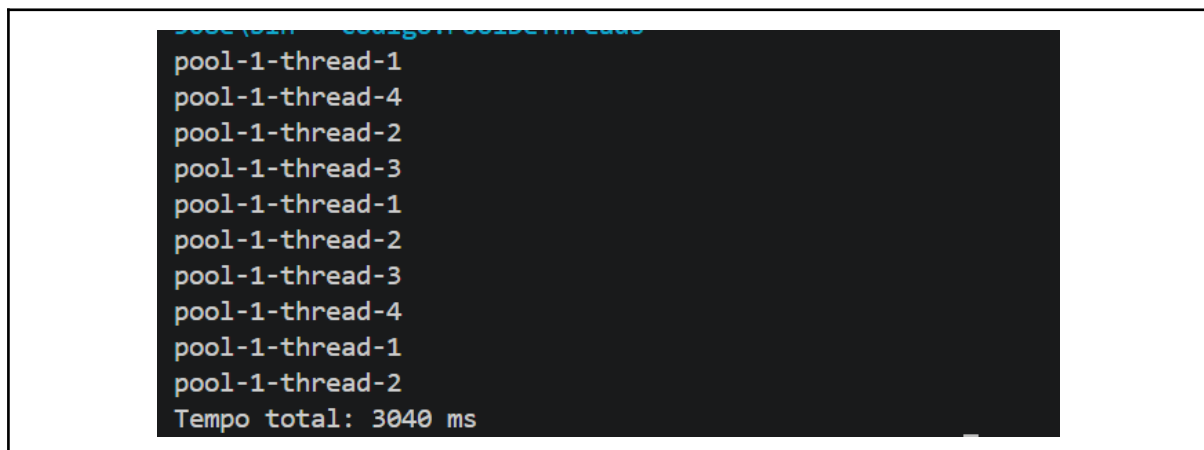
Ao criar um objeto comum em Java ele fica armazenado na memória da própria JVM, sem a necessidade de pedir nada de especial para o SO. Já para criar uma thread nativa, o SO precisa reservar uma pilha de memória própria para essa thread, além de criar estruturas de controle dentro do kernel, para que o escalonador do sistema saiba que essa thread existe e precisa ser executada em algum momento. Assim, criar uma thread é uma operação mais cara porque ela sai da JVM e mexe em recursos do sistema operacional, que são mais pesados e mais lentos de alocar do que um simples objeto Java.

Parte C: O que esse limite sugere sobre usar 1 thread por requisição em um servidor web?

Sugere que esse modelo não escala bem, porque se um servidor web cria uma thread de SO nova para cada requisição que chega, então, com um número alto de requisições simultâneas o servidor irá cair. Por exemplo, se um servidor receber 100.000 acessos ao mesmo tempo, ele tentará criar 100.000 threads. Como o SO costuma atingir seu limite e falhar ao passar de 50.000 threads, o servidor web lançará o erro de falta de memória e cairá.

Parte D: Com 4 threads atendendo 10 clientes, o tempo total ficou perto de 1s, 2s ou 3s?

Ficou perto de 3s, conforme ilustrado pela figura 2. As 4 threads conseguem executar 4 tarefas ao mesmo tempo, e como são 10 tarefas no total, foram necessárias 3 rodadas de execução simultânea: a primeira e a segunda com as 4 threads executando 4 tarefas, e a última com 2 threads executando as 2 tarefas restantes. Como cada rodada levou cerca de 1s pra executar, o tempo total foi de aproximadamente 3s.

A terminal window with a dark background and light-colored text. It shows a list of thread names: 'pool-1-thread-1', 'pool-1-thread-4', 'pool-1-thread-2', 'pool-1-thread-3', 'pool-1-thread-1', 'pool-1-thread-2', 'pool-1-thread-3', 'pool-1-thread-4', 'pool-1-thread-1', and 'pool-1-thread-2'. At the bottom, it displays 'Tempo total: 3040 ms'.

```
pool-1-thread-1  
pool-1-thread-4  
pool-1-thread-2  
pool-1-thread-3  
pool-1-thread-1  
pool-1-thread-2  
pool-1-thread-3  
pool-1-thread-4  
pool-1-thread-1  
pool-1-thread-2  
Tempo total: 3040 ms
```

Figura 2

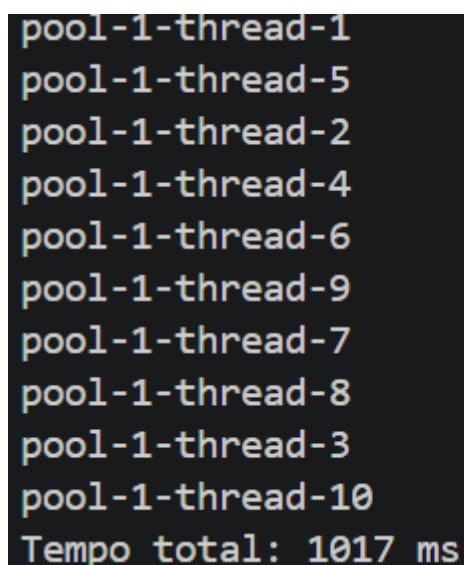
Parte E: Uma Virtual Thread é uma thread de Sistema Operacional? Se não, o que ela é?

Não, ela é uma estrutura de software criada, mapeada e gerenciada inteiramente pela JVM. Por isso ela consegue ser mais leve que threads de SO, pois ela não carrega o mesmo custo de memória e de estruturas de kernel que uma thread nativa carrega. Isso acontece porque o SO não enxerga nem gerencia as Virtual Threads. Do ponto de vista dele existem apenas as poucas threads reais, as carrier threads, que a JVM alocou. É a JVM quem decide quando uma Virtual Thread pega uma carrier thread para rodar de verdade, e quando ela solta essa carrier thread porque está esperando algo.

Exercícios de Fixação

1) Trocar o pool fixo por `newCachedThreadPool()` — o comportamento muda?

Sim, o tempo total de execução fica de aproximadamente 1s, para as mesmas 10 tarefas da Parte D, mas diferente da Parte D onde o tempo total foi de cerca de 3s. Pesquisando sobre esse método, descobri que isso acontece porque ele não define um número fixo de threads, criando uma nova thread para cada tarefa que chega, priorizando o paralelismo. Assim, elas não ficam esperando para serem executadas, o que acontecia na Parte D com o número fixo de threads, e por isso o tempo total de execução diminuiu.

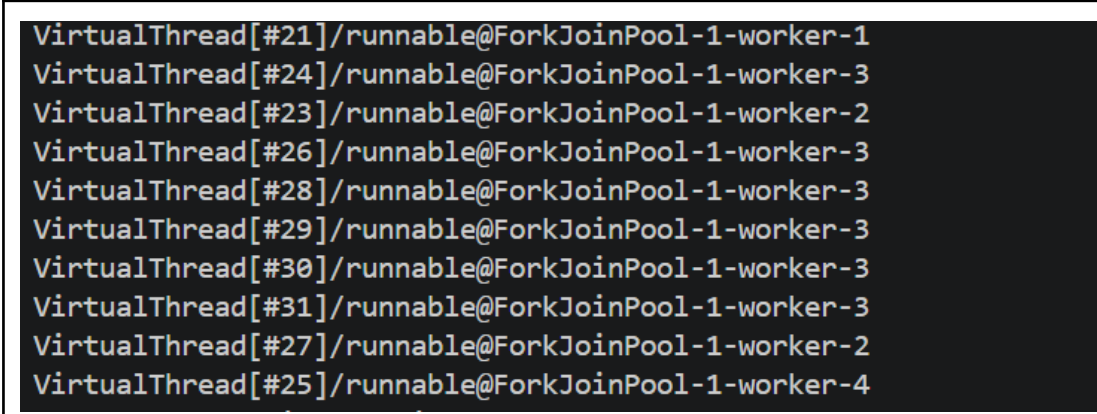


```
pool-1-thread-1  
pool-1-thread-5  
pool-1-thread-2  
pool-1-thread-4  
pool-1-thread-6  
pool-1-thread-9  
pool-1-thread-7  
pool-1-thread-8  
pool-1-thread-3  
pool-1-thread-10  
Tempo total: 1017 ms
```

Figura 3

2) Imprimir Thread.currentThread() na Parte E — é uma VirtualThread?

Sim, Thread.currentThread() retorna uma VirtualThread, conforme ilustrado pela figura 4, onde aparece a saída (VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1). Além disso, o terminal evidencia que, apesar de existirem várias Virtual Threads diferentes, todas elas são executadas por um número pequeno de carrier threads reais (ForkJoinPool-1-worker-1 a worker-4), confirmando na prática o mecanismo de carrier threads.

A terminal window with a dark background and light-colored text. It displays ten lines of output, each representing a VirtualThread instance. The format of each line is 'VirtualThread[#ID]/runnable@ForkJoinPool-1-worker-N', where #ID is a number and N is a worker ID. The worker IDs are 1, 3, 2, 3, 3, 3, 3, 3, 2, and 4, respectively, showing that multiple VirtualThreads are multiplexed onto a small set of worker threads.

```
VirtualThread[#21]/runnable@ForkJoinPool-1-worker-1
VirtualThread[#24]/runnable@ForkJoinPool-1-worker-3
VirtualThread[#23]/runnable@ForkJoinPool-1-worker-2
VirtualThread[#26]/runnable@ForkJoinPool-1-worker-3
VirtualThread[#28]/runnable@ForkJoinPool-1-worker-3
VirtualThread[#29]/runnable@ForkJoinPool-1-worker-3
VirtualThread[#30]/runnable@ForkJoinPool-1-worker-3
VirtualThread[#31]/runnable@ForkJoinPool-1-worker-3
VirtualThread[#27]/runnable@ForkJoinPool-1-worker-2
VirtualThread[#25]/runnable@ForkJoinPool-1-worker-4
```

Figura 4

3) Explicar processo x thread com a analogia do guichê de atendimento

Nessa analogia, é como se o processo fosse uma agência, com seu prédio, seus sistemas e seus recursos próprios, isolados de outras agências. Já as threads seriam os atendentes que trabalham nessa agência, atendendo os clientes, que seriam as tarefas, de forma paralela. Além disso, os atendentes compartilham os mesmos sistemas, recursos e informações, já que estão todos trabalhando na mesma agência, da mesma forma como as threads compartilham a memória do processo. Isso faz com que todos os atendentes sejam prejudicados caso um atendente manipule as informações da agência de forma errada, assim como as threads podem afetar o trabalho umas das outras ao manipularem as variáveis do processo. E contratar um novo atendente é mais barato do que construir uma agência nova, da mesma forma como construir um novo processo exige mais recursos do que criar uma thread nova.

4) Escolher, com justificativa, a abordagem ideal para um servidor com milhares de conexões

A melhor abordagem seria a Virtual Threads, porque nela se cria uma thread leve, gerenciada pela JVM para cada conexão, sem depender de um pool

fixo, evitando assim a fila de espera. Além disso, como ela não exige a pilha de memória e as estruturas de kernel de uma thread nativa, ela é muito mais econômica. E também, poucas carrier threads atendem um número enorme de Virtual Threads, já que cada uma só ocupa uma carrier thread enquanto executa de fato, liberando-a quando está apenas esperando alguma coisa. Isso permite escalar para milhares de conexões sem esgotar a memória do sistema, como aconteceria com threads de SO tradicionais.